WISEConv: A Worklist-Driven Masked-Convolution Runtime for GPUs

Anonymous Author(s)

Abstract

Masked convolution promises to cut the per-frame inference latency of vision models by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to convert sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime that derives an active-output worklist from the upstream mask and uses it to drive the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by restricting computation to active positions while retaining the regular dense datapath. To amortize construction overhead and keep the compute pipeline fully utilized despite per-frame-varying worklist lengths, WISEConv co-designs construction and consumption: compatible operators reuse the constructed worklist without reconstruction, the worklist emission order preserves tile-local spatial reuse, and a data-driven adaptive decomposition adjusts parallel work to the activity level without host synchronization. Across four perception models and six GPU platforms (four discrete, two embedded), WISEConv reduces full-model latency by 22.1–46.5% relative to the fastest competing path and per-frame energy by 28.0–59.6% on the embedded platforms.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates the per-frame inference latency of many latency-critical vision tasks [2, 9, 37], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks active positions, and the operator computes only those. The mask may derive from event-camera increments [7, 22], temporal differences [16, 27], or learned spatial gates [20, 34]. Regardless of source, it poses one operator-level problem: compute the convolution only at the active positions of the dense feature grid.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates share overlapping receptive fields, yielding predictable memory access and data reuse. However, unlike static sparsity in matrix multiplication, masked convolution selects positions at runtime per frame, requiring the operator to access scattered, irregularly-overlapping neighborhoods and handle a varying workload, conflicting with tiled execution.

Existing masked-convolution systems preserve this regularity or eliminate wasted work, but not both. *Tile skipping* [7, 16, 27, 29] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [20, 34, 38] (as an execution path) instead extracts active positions exactly, materializes receptive fields, and scatters the computation results to the dense grid, sacrificing regularity and incurring extraction, irregular access, and writeback overhead.

We characterize this trade-off along two axes (§2.2): *throughput* (operations per second) and *useful-compute ratio* (the fraction of issued operations that produce needed outputs). In our sparsest workload (the DynConv-gated model, §5.2), the masks render 84.4% of the convolution work unnecessary, yet neither approach turns this into proportional latency reduction. Tile skipping [27] suffers a useful-compute ratio of only 49.1% and cuts latency by at most 40.6%. Gather-scatter drives the useful-compute ratio near one but sustains only 9.3% of tile skipping's throughput, running 3.1× slower than the dense path.

Our key insight is that position-level selectivity does not require abandoning the dense convolution pipeline. A dense tile already derives its reads and writes from output coordinates, so replacing its contiguous coordinate block with an active-output worklist changes which outputs are computed without changing the per-position reduction, the dense tensor layout, or the weight layout. With this substitution, the operator runs as a dense tiled pipeline, raising the useful-compute ratio toward one without materializing input patches.

Translating this into latency reduction requires handling two challenges. First, *construction efficiency*: construction scans the mask, so its cost scales with the layer's spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction's share of latency therefore grows as activity falls and channels thin (while enough positions remain active to keep the pipeline full), and accumulates across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist's order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full, and because the length is device-resident, any host-side decision that depends on it

costs a synchronization on the per-frame path. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. Construction is cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it. Consumption is dense-like: construction emits contiguous per-tile segments ordered by the dense tile traversal, giving the compute stage tile-local structure without global sorting. Compute then decomposes parallel work adaptively according to device, layer shape, and activity, choosing each operator's configuration from offline calibration. For temporally correlated inputs, it tracks activity across frames and sustains GPU utilization without host synchronization.

This paper makes the following contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, exposing why existing work does not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across four discrete GPUs and two Jetson platforms. WISEConv reduces full-model latency relative to the fastest baseline by 34.2–46.5% on the discrete GPUs and 22.1–38.6% on the Jetson platforms, and cuts per-frame energy over dense convolution by 28.0–59.6% on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [2, 31, 35, 37, 41], where per-frame inference latency bounds system responsiveness [8–10, 17]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [7, 22], temporal residuals between video frames [3, 16, 26, 27, 36], or learned input-dependent gates [12, 20, 34, 38]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the tensors and arithmetic of dense convolution and restricts only which output coordinates it computes. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$. The active ratio $\alpha = \|M_{out}\|_0 / |\mathcal{G}|$, the active fraction of the grid, is the intrinsic sparsity the policy exposes.

An execution path issues computation over a sequence of position slots C, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i \, \mathbf{x}[\mathcal{N}_i(p)], \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. Positions the path does not compute retain their existing value in $\mathbf{y}$: masked convolution updates the output in place, so $\mathbf{y}[p]$ for $p \notin \{c \in C : c \neq \perp\}$ carries over from the previous frame (event and temporal policies) or from a one-time dense initialization (spatial gating). Coverage is therefore the correctness condition: every position the policy marks as changed is recomputed, and every other position is already correct by construction. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and the required work, while C is determined by the path.

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [7, 16, 21, 26, 29, 36] inherits the dense pipeline's tile-based computation, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [27] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [20, 38] instead extracts exactly the active coordinates, computes, and scatters them back,

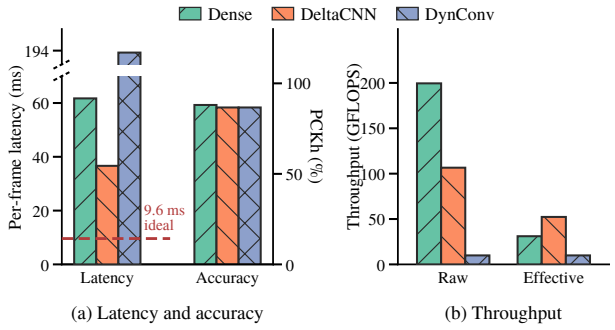

Figure 1. Statistics of DynConv [34], a pose estimator with learned spatial gating, on the MPII human-pose dataset [1] and Jetson AGX Orin, comparing the native cuDNN path (Dense) against the two sparse paths. (a) End-to-end average per-frame latency (left axis; the dashed line marks the dense latency scaled by the active ratio aggregated across layers) and PCKh accuracy (right axis; PCKh evaluates the percentage of keypoints localized within half the head size). The sparse paths apply the same mask, so their accuracy coincides. (b) Raw throughput T and effective throughput T_{eff}, aggregated across the model's convolution layers. The dense useful-compute ratio equals the active ratio set by DynConv's spatial gating ($\alpha \approx 15.6\%$).

removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [34] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

We measure each path by two quantities. Its useful-compute ratio is the fraction of issued position slots (including $\perp$) that carry an active output,

$$\eta = \frac{\|M_{out}\|_0}{|C|} \le 1, \tag{3}$$

defined for $|C| > 0$; an operator whose mask is empty issues no slots and carries no ratio. Its throughput is the rate at which it processes issued FLOPs,

$$T = \frac{2\,|C|\,k^2 C_{in} C_{out}}{t}, \tag{4}$$

where t is the per-frame latency, including all auxiliary work the path performs to enable computation (mask processing, coordinate generation, auxiliary passes). Because $|C|$ counts only the convolution's issued position slots, auxiliary costs reduce T without changing η. The leading factor of 2 counts each multiply-accumulate as two FLOPs. For the required $2\,\|M_{out}\|_0\,k^2 C_{in} C_{out}$ FLOPs, latency is set by the **effective throughput** $T_{\text{eff}} = \eta T$, the rate at which a path computes needed outputs.

Figure 1 traces this on Jetson AGX Orin for the sparsest workload in our evaluation. The policy marks 84.4% of the convolution work unnecessary, and the sparse paths discard it with negligible accuracy loss, so the remaining work sets a 9.6 ms latency floor (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [27]) reaches 36.6 ms, still 3.8× the floor, and gather-scatter, even with DynConv built to amortize its overhead, runs at 193.5 ms, far slower than dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput near half of dense but at a useful-compute ratio of only 49.1%; gather-scatter lifts the useful-compute ratio near one, yet extraction and data movement collapse its throughput to 5.0% of dense (Fig. 1b). Neither path efficiently converts the GPU's capability into effective throughput for masked convolution.

2.3 Distinction from Related Sparse Execution

Related sparse-execution work differs from WISEConv's masked convolution in representation, operator, sparsity type, or platform. 3D sparse convolution (SpConv v2 [30], TorchSparse++ [32, 33], Minuet [39], MinkowskiEngine [6], submanifold sparse convolution [15]) is built to organize computation over an unordered list of active voxels, constructing coordinate maps that route irregular input coordinates to outputs; its input is sparse by construction rather than a dense grid under a runtime mask, so it is related infrastructure rather than a baseline. Sparse matrix multiplication (DTC-SpMM [11], MaxK-GNN [28], Sputnik [13]) reorganizes a sparse matrix operand so the dense datapath can consume it; its sparsity lives in the matrix structure of a different operator, not in a spatial mask over a convolution grid. Structured weight sparsity (2:4 tensor cores [24]) and sparse-format compilers (TACO [19], SparseTIR [40]) exploit a static structural sparsity fixed at compile time in the weights or a known format, rather than a mask discovered at runtime over the spatial grid. DPACS [14] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

3 WISEConv Design

3.1 Overview

WISEConv drives the position dimension of a dense tiled convolution from an active-output worklist. A dense tile already derives its input addresses and output locations from output coordinates, so WISEConv changes which coordinates enter the pipeline and leaves each position's channel and kernel reduction untouched. WISEConv therefore obtains position-level selectivity without giving up the dense tensor layout, the weight layout, or the regular reduction.

The operator runs in two stages that exchange only metadata. The *construction stage* converts the upstream mask into the output mask M_{out}, a worklist $\mathcal{W}$ of scheduled output coordinates, and a length $n_{\mathcal{W}}$ that stays in device memory. The *compute stage* reads coordinates from $\mathcal{W}$ and drives a tiled convolution from them. Fresh construction schedules exactly

the positions marked by M_{out}; along a compatible operator sequence, a later operator may instead inherit a worklist that schedules a superset of its own active set (§3.2.2). Figure 2 shows the two stages and the metadata they exchange.

The two stages control separate factors of the useful-compute ratio. On the worklist path, compute issues positions in tiles of B_M entries (the position-tile width, §3.3.1), so the slot sequence C of §2.1 has length $B_M\lceil n_{\mathcal{W}}/B_M\rceil$, with trailing slots $\perp$. Eq. 3 then factors as

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{cov}}\ \text{(schedule tightness)}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{B_M\lceil n_{\mathcal{W}}/B_M\rceil}}_{\eta_{\text{pack}}\ \text{(position-axis packing)}} \tag{5}$$

for $n_{\mathcal{W}} > 0$, the case where compute issues slots at all. η_{cov} measures how tightly the worklist covers M_{out}: construction and the reuse policy set $n_{\mathcal{W}}$ and therefore fix it. η_{pack} measures how efficiently worklist entries fill position tiles: the compute decomposition sets B_M and therefore fixes it for a given $n_{\mathcal{W}}$. Throughput T captures the remaining costs, including construction time, indirect input access, underfill, and any synchronization the operator performs. Accordingly, WISEConv first reduces construction cost and amortizes what remains (§3.2), then organizes the compute stage to consume variable-length worklists at high throughput without waiting on the host (§3.3).

3.2 Construction Efficiency and Amortization

3.2.1 Tile-Local Worklist Construction. Producing a tight worklist requires at least one scan of the output grid, and the coordinates alone do not make the result efficient to consume. A globally stable compaction retains the full dense traversal order, but computing global offsets costs a device-wide prefix scan or a second pass. Appending active coordinates independently completes in one pass, but leaves an arbitrary coordinate order that discards the spatial grouping compute benefits from. Extraction-based construction is more expensive still: it materializes receptive-field feature data alongside the coordinates, a volume that grows with the active count and with C_{in}.

WISEConv needs neither ordering extreme: compute benefits when nearby coordinates stay grouped, but does not require a global total order over the whole worklist. Construction therefore emits segments that are globally unordered and locally ordered, in one pass over mask and coordinate metadata. Before the pass begins, WISEConv pre-allocates $\mathcal{W}$ to the output-grid upper bound $|\mathcal{G}| = H_{out}W_{out}$ entries and zeroes a device-resident counter. The pass partitions the output grid into construction tiles τ of $B_\tau \times B_\tau$ positions and, for each tile, tests whether each output position is active, writes M_{out}, and counts the tile's local active positions n_τ. Each tile reserves an interval $[\text{base}, \text{base} + n_\tau)$ from the counter, then writes its active coordinates into that interval in the tile's dense traversal order (Algorithm 1). Once all tiles finish, the counter holds the valid worklist length $n_{\mathcal{W}}$. Worklist capacity and launch geometry follow from the static layer geometry, so this pass needs no host readback of an active count, no dynamic allocation, and no device-wide stable offsets.

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in}, output grid $\mathcal{G}$, construction tile size B_τ
Ensure: Worklist $\mathcal{W}$, output mask M_{out}, worklist length $n_{\mathcal{W}}$

```
n_W ← 0
for each spatial tile τ of B_τ × B_τ positions in parallel do
    for each position p ∈ τ in parallel do
        active(p) ← ∃ r ∈ N(p) : M_in[r] = 1
        M_out[p] ← active(p)
    end for
    n_τ ← |{p ∈ τ : active(p)}|
    base ← AtomicReserve(n_W, n_τ)
    Write active p ∈ τ to W[base, base+n_τ) in traversal order
end for
```

Segmented construction still produces a tight schedule. Every output position belongs to exactly one construction tile, so each active coordinate is emitted exactly once. The counter hands out disjoint intervals whose sizes sum to the active count, so the emitted active coordinates fill the valid prefix with no gap and no duplicate, giving

$$n_{\mathcal{W}} = \|M_{out}\|_0, \qquad \eta_{\text{cov}} = 1.$$

Tiles may reserve their segments in any order, because the coverage contract (§2.1) requires only that each active coordinate appear in the valid worklist prefix, not that segments follow a prescribed order. Traversal order within a segment is a performance property and carries no correctness role.

Local segment order is what construction gives compute in place of a global order. All coordinates in a segment come from one bounded spatial tile and keep that tile's dense traversal order, so adjacent entries within a segment address nearby receptive fields, while entries that straddle a segment boundary carry no such guarantee. Grouping that survives inside a segment is the part compute uses, and it costs one atomic reservation per tile rather than the device-wide offsets a global order would need. A compute tile may span several segments, so the locality a tile actually sees depends on segment length relative to B_M rather than on any global-order guarantee. As activity falls, segments tend to shorten and one compute tile draws from more of them.

Construction scales with spatial geometry and kernel footprint, and does not scale down with activity or channel width. For a mask that propagates along receptive fields, the pass performs $O(H_{out}W_{out}k^2)$ mask and coordinate work and

[TODO] WISEConv overview. Left: dense input feature tensor and input mask. Middle: construction emits the output mask M_{out}, the worklist coordinates, and the device-resident worklist length; a compatible operator inherits this metadata instead of rebuilding it. Right: worklist coordinates replace the contiguous position source of tiled convolution, while channel and kernel reduction proceed as in the dense path. Bottom: the asynchronous input-activity observation selects the compute configuration only.

Figure 2. The WISEConv two-stage pipeline. Construction discovers scheduled output coordinates, compacts local segments into the worklist, and leaves the worklist length in device memory; a compatible operator inherits this metadata instead of rebuilding it. Compute drives a tiled convolution from the worklist while retaining regular channel and kernel reduction. An asynchronous input-activity observation selects the compute configuration and never gates execution.

reads neither feature nor weight tensors,[1] For a freshly constructed worklist in a standard ungrouped convolution, the compute it enables costs $O(\alpha H_{out} W_{out} k^2 C_{in} C_{out})$. Grouping changes the channel factor but preserves the asymmetry: construction stays independent of channel width and does not shrink with the compute it schedules. Construction's share of operator latency therefore grows precisely where selective compute becomes cheapest, on low-activity and thin-channel operators. Repeating this scan at every same-grid operator would accumulate the same channel-independent cost across a block, which motivates amortizing construction across compatible operator sequences.

3.2.2 Cross-Operator Worklist Reuse. Many same-grid operators between receptive-field-changing layers can execute on the worklist their predecessor already built. A block interleaves the spatial convolutions that change the receptive field with 1×1 convolutions, activations, and inference-time normalization. These operators cannot introduce an active coordinate outside the incoming schedule: they either preserve the active set or narrow it to a subset. Reconstructing the worklist would therefore either reproduce the incoming schedule or merely tighten an already legal superset schedule, while repeating the same channel-independent grid scan. WISEConv accordingly defines the amortization unit for construction as the compatible operator sequence, not the individual operator.

[1]When the upstream policy produces the output active set directly (learned spatial gating [20, 34]), construction replaces receptive-field propagation with the gate's own activity check, removing the k^2 factor from that layer's construction cost.

WISEConv separates the semantic active set from the execution schedule. This separation enables safe inheritance. Four items travel together across operator boundaries: the feature tensor, the output mask, the worklist buffer, and the device-resident worklist length. Write operator i's active set and the scheduled coordinate set of the worklist it consumes as

$$\mathcal{A}^i = \{p : M^i_{out}[p] = 1\}, \quad \mathcal{S}^i_{\mathcal{W}} = \{\mathcal{W}[m] : 0 \le m < n_{\mathcal{W}}\}.$$

Construction writes each coordinate once, so $|\mathcal{S}^i_{\mathcal{W}}| = n_{\mathcal{W}}$, and reuse forwards the buffer untouched and preserves this. Whenever the coordinates in $\mathcal{S}^i_{\mathcal{W}}$ still name the downstream output coordinate space and satisfy $\mathcal{A}^i \subseteq \mathcal{S}^i_{\mathcal{W}}$, the coverage contract of §2.1 holds for operator i, which then consumes the incoming worklist and runs no construction pass of its own. The mask M^i_{out} records the semantic active set the upstream policy requires; the inherited worklist records the coordinates compute actually schedules, which under a superset schedule may strictly contain the semantic active set. That single inclusion defines every reuse case.

An operator that preserves the active set inherits the incoming schedule tightness unchanged. Two classes qualify. A same-grid 1×1 convolution with unit stride, zero padding, and unit dilation maps each input position to one output at the same coordinate. Active-set-preserving pointwise operators, among them non-thresholding activations and inference-time normalization, map each position to itself and preserve each position's active flag. Because $\mathcal{A}^i = \mathcal{A}^{i-1}$ and $n_{\mathcal{W}}$ is unchanged, $\eta^i_{\text{cov}} = \eta^{i-1}_{\text{cov}}$, and WISEConv removes one construction pass at no cost in schedule tightness.

An operator that narrows the active set trades tightness for one fewer pass. It still satisfies $\mathcal{A}^i \subseteq \mathcal{S}^i_{\mathcal{W}}$ with $\eta^i_{\text{cov}} = |\mathcal{A}^i|/n_{\mathcal{W}}$. A thresholding activation reduces this ratio below its predecessor's. WISEConv updates M^i_{out} and still forwards the incoming worklist, so compute runs a legal superset schedule. The extra scheduled coordinates buy one fewer $O(H_{out}W_{out})$ construction pass, and the pointwise narrowing operators considered here have unit kernels, so the pass they avoid carries no k^2 factor. Because preserving operators hold tightness and narrowing operators lower it, η_{cov} is non-increasing along a compatible sequence and returns to 1 only at a rebuild.

Rebuilding is a question of schedule validity, not of accumulated looseness. When the downstream output coordinate space changes, or when a $k > 1$ receptive field may expand the active set beyond $\mathcal{S}^i_{\mathcal{W}}$, the inherited worklist no longer satisfies the coverage contract. WISEConv runs the pass of §3.2.1 at these boundaries, recovering $\mathcal{A}^i = \mathcal{S}^i_{\mathcal{W}}$ and $\eta^i_{\text{cov}} = 1$, and starts the next compatible sequence from that tight schedule. Whether the receptive field actually adds coordinates is data-dependent, so WISEConv rebuilds whenever such expansion is possible. Reuse therefore trades schedule tightness for one fewer construction pass. WISEConv fixes this decision from operator semantics rather than measuring profitability at run time, avoiding a per-operator activity observation and a data-dependent dispatch between two execution paths; §5.4 evaluates whether the removed construction cost outweighs the additional scheduled work.

Across a block, construction therefore scales with rebuild points rather than operator count. The same-grid operators between receptive-field-changing convolutions inherit the leading worklist, while each rebuild starts a new tight sequence. Each compatible sequence consequently delivers one tile-locally structured, device-resident worklist to the compute stage, whose length varies across frames and layers.

3.3 End-to-End Worklist Efficiency

3.3.1 Persistent Worklist-Driven Compute. Driving convolution from a worklist turns the position dimension of the underlying GEMM from a layer-shape constant into a device-resident runtime quantity. In a batch-1 dense convolution, an implicit GEMM maps output positions onto a row dimension M, output channels onto a column dimension N, and the channel and kernel reduction onto a depth K:

$$M = H_{out}W_{out}, \qquad N = C_{out}, \qquad K = k^2 C_{in}.$$

One tile computes $B_M \times B_N$ output elements and walks the full K dimension in B_K-wide steps. All three extents follow from the layer shape, so the backend selects one tile decomposition (B_M, B_N, B_K) ahead of execution. WISEConv leaves N and K unchanged and sets $M = n_{\mathcal{W}}$, a row extent that varies per invocation and that the host does not learn once construction leaves it on the device. Two separate problems follow: how to launch and complete the scheduled work when the host cannot observe the row extent, and how to choose the decomposition that depends on it. This subsection solves the first, §3.3.2 the second.

WISEConv substitutes only the coordinate source of the row dimension. The dense path maps row m to the m-th contiguous output coordinate, WISEConv maps it to $\mathcal{W}[m]$:

$$p(m) = \begin{cases} p_m, & \text{dense path,} \\ \mathcal{W}[m], & \text{WISEConv.} \end{cases} \tag{6}$$

Each block reads B_M coordinates from the worklist, decodes their receptive-field base addresses, and computes the corresponding $B_M \times B_N$ outputs. Each worklist coordinate determines both the receptive-field loads and the output store. The kernel gathers input values directly from the dense feature tensor while retaining the standard B_K-tiled reduction over K, materializing neither an im2col matrix nor an extracted-patch buffer. Construction's tile-local segments supply the local spatial structure those gathers exploit. WISEConv then launches an occupancy-sized persistent grid, sized at initialization from how many blocks of the chosen configuration can co-reside on an SM, so its dimensions never depend on $n_{\mathcal{W}}$. At kernel entry, blocks read the current $n_{\mathcal{W}}$ from device memory and compute the work-item count

$$n_{\text{base}} = \left\lceil \frac{n_{\mathcal{W}}}{B_M} \right\rceil \left\lceil \frac{C_{out}}{B_N} \right\rceil, \tag{7}$$

then consume all position and channel tiles through a grid-stride loop (Algorithm 2). The host launch never consults the worklist length, while the device-resident count sets the loop bound, the last-tile padding, and the zero-work early return. Grouped and depthwise convolutions use the same interface, each group forming an independent GEMM with proportionally smaller N and K: the work-item count gains a factor of the group total, and the groups divide C_{out} among them.

Two more direct routes to a device-resident row extent each carry a cost the persistent grid avoids. Reading $n_{\mathcal{W}}$ back to the host to dimension the grid inserts a device-to-host synchronization at every operator on the per-frame path. Submitting blocks for the full output-grid upper bound removes that synchronization but ties the submitted block count to the full output grid, so most blocks find no work once the worklist is short. WISEConv pays neither: the submitted block count tracks device residency, while the work bound is resolved on the device. Persistent execution therefore resolves the device-resident problem size, but it does not determine which decomposition best serves that size.

3.3.2 Adaptive Workload Decomposition. The best candidate for a given worklist length is measurable offline. Let $q \in \mathcal{Q}_i$ denote a candidate for operator i, with $\mathcal{Q}_i$ enumerated once on the deployment device. A candidate is either a worklist tiling (B_M, B_N, B_K, S_k), whose entries set the position tile, the channel tile, the reduction step, and a split-K degree, or,

Algorithm 2 Persistent Worklist-Driven Implicit GEMM ($S_k = 1$)

Require: Device-resident worklist $\mathcal{W}$ and its length, input $\mathbf{x}$, weights $\mathbf{w}$
Ensure: Output $\mathbf{y}$ (updated at worklist coordinates)

```
n_W ← read worklist length from device {dense: M = H_out W_out is host-known}
n_base ← ⌈n_W / B_M⌉ × ⌈C_out / B_N⌉ {shrinks with a short worklist}
for tile = block_id to n_base step grid_size do
    (b_m, b_n) ← decompose tile {persistent grid-stride scheduling}
    p_j ← W[b_m B_M + j] for j < B_M, or ⊥ past n_W {dense: p_j = b_m B_M + j}
    Decode each non-⊥ p_j to its receptive-field base address
    Initialize accumulator Acc ← 0_{B_M × B_N}
    for each B_K-wide reduction step over K = k^2 C_in do
        x_tile ← gather from x at receptive-field addresses {reduction identical to dense}
        w_tile ← load corresponding weight slice
        Acc ← Acc + x_tile · w_tile
    end for
    Store Acc to y at the non-⊥ coordinates p_j
end for
```

where the operator's semantics permit, the dense cuDNN path. Write the normalized worklist length of operator i as

$$\lambda_i = \frac{n^i_{\mathcal{W}}}{|\mathcal{G}_i|} = \frac{\alpha_i}{\eta^i_{\text{cov}}}, \tag{8}$$

which follows from $\alpha_i = |\mathcal{A}^i|/|\mathcal{G}_i|$ and $\eta^i_{\text{cov}} = |\mathcal{A}^i|/n^i_{\mathcal{W}}$. Fresh construction gives $\lambda_i = \alpha_i$; superset reuse lowers η^i_{cov} and therefore puts λ_i above α_i. An index ℓ discretizes λ_i into a small set of levels, and $n_{i,\ell}$ denotes the corresponding representative absolute length for operator i. At that length,

$$q^*_{i,\ell} = \arg\min_{q \in Q_i} \widehat{t}_i(q;\, n_{i,\ell}), \tag{9}$$

where $\widehat{t}_i$ is measured candidate latency. Indexing λ_i rather than α_i matters because the scheduled length is what compute pays for. The candidate set covers both ends of the length range. Smaller B_M and split-K expose more parallel work for short worklists, the former by bounding the trailing-tile padding that η_{pack} measures (Eq. 5), the latter by multiplying the work-item count of Eq. 7 by S_k. The dense candidate, the p_m branch of Eq. 6, covers the near-dense regime. Calibration charges each candidate for all required auxiliary work, including split-K output zeroing and atomic accumulation, and excludes the construction stage every candidate shares. Its worklists preserve construction's tile-local grouping, so $\widehat{t}_i$ captures the runtime indirect-access behavior. Offline selection is therefore routine; obtaining its runtime index is not. The index depends on the current device-resident $n_{\mathcal{W}}$, whose synchronous readback at every operator would undo the persistent design.

WISEConv separates the current execution bound from the configuration signal. Persistent compute always reads the current $n_{\mathcal{W}}$ on the device; the host predicts each operator's level from a lagged input-activity observation alone. Let M^f_0 be the network-input mask of frame f over the grid $\mathcal{G}_0$, the single mask from which the propagated layer masks derive rather than the input mask of an individual convolution. Its activity and that activity's discretization are

$$\beta_f = \frac{\|M^f_0\|_0}{|\mathcal{G}_0|}, \qquad b_f = \text{Bucket}(\beta_f).$$

A per-operator mapping $\mathcal{M}_i$ turns such a bucket into a level for operator i, and the calibration table turns that level into a candidate:

$$\mathcal{L}_i[\ell] = q^*_{i,\ell}, \qquad q_i(f) = \mathcal{L}_i\Big[\mathcal{M}_i(\hat{b}_f)\Big], \tag{10}$$

where $\hat{b}_f$ is the bucket of the most recent input-activity observation to have reached the host by the start of frame f, possibly from an earlier frame. An approximate, lagged signal thus selects the decomposition, while the device-resident count keeps setting the execution bound.

A second offline pass fits $\mathcal{M}_i$ from representative traces. It runs the model with its deployment reuse decisions already fixed, records each frame's network-input bucket b_f, and records the length $n^i_{\mathcal{W}}(f)$ every operator actually consumes, fresh or inherited, rather than active counts derived only from M^i_{out}. Converting those lengths through Eq. 8 and taking the per-bucket median gives

$$\mathcal{M}_i(b) = \text{Level}\Big(\underset{f\,:\,b_f = b}{\text{median}}\, \lambda_i(f)\Big), \tag{11}$$

a representative rather than a worst-case level, so a few outlying frames do not move an operator off the configuration that serves its usual length. WISEConv fits one $\mathcal{M}_i$ per operator, because output-grid resolution, receptive-field size, and mask-propagation depth amplify the same input activity differently at different depths. Two workload properties make this global, lagged observation informative. The policy supplies an input mask whose active positions propagate along receptive fields (Eq. 1), which ties global input activity to the per-layer lengths. Successive inputs also correlate in time, so an earlier observation predicts the current frame. Our event and temporal workloads propagate masks from a shared network input and show sufficient temporal correlation on representative traces; §5.3 evaluates the resulting selection quality. The two offline passes therefore divide cleanly: Eq. 9 maps a worklist-length level to the fastest candidate, and Eq. 11 maps an input-activity bucket to a representative level.

WISEConv supplies $\hat{b}_f$ through a completed-only, non-blocking feedback loop (Algorithm 3). Each operator starts

Algorithm 3 Trace-Calibrated Nonblocking Configuration Selection

Require: Per-level candidate tables $\mathcal{L}_i$ from Eq. 9
Ensure: Per-operator candidates $q_i(f)$, chosen without host-device synchronization

Offline (representative traces, deployment reuse decisions fixed):
for each frame f of the traces **do**
  $b_f \leftarrow \mathrm{Bucket}(\|M_0^f\|_0/|\mathcal{G}_0|)$
  for each operator i **do**
    append $\lambda_i(f) = n^i_{\mathcal{W}}(f)/|\mathcal{G}_i|$ to samples $R_i[b_f]$ {length actually consumed}
  end for
end for
$\mathcal{M}_i[b] \leftarrow \mathrm{Level}(\mathrm{median}\, R_i[b])$ for all i, b
Online (per sequence):
$q_i \leftarrow$ calibrated default entry of $\mathcal{L}_i$ for all i
for each frame f **do**
  if an observation has completed **then**
    $\hat{b}_f \leftarrow$ bucket of the most recent completed observation
    $q_i \leftarrow \mathcal{L}_i[\mathcal{M}_i(\hat{b}_f)]$ for all i
  else
    retain current q_i {never blocks on a pending transfer}
  end if
  generate M_0^f; enqueue asynchronous transfer of $\|M_0^f\|_0$, dropping it if the buffer is full
  run construction and compute with q_i, each operator bounded by its current device-resident $n^i_{\mathcal{W}}$
end for

a sequence from a calibrated default entry, so a configuration is defined before any observation arrives. Frame f then runs in a fixed order. The runtime first polls for observations that have already completed and, if one is available, reads $\mathcal{L}_i$ at the level the most recent observation implies. If none has completed, every operator keeps its current configuration. WISEConv then generates the input mask and enqueues an asynchronous transfer of the resulting active count, after which construction and compute launch. Polling therefore never waits on the transfer it just issued, and a full observation buffer drops the new count rather than blocking. Completed-only polling, reuse of the current configuration, and drop-on-full buffering ensure that configuration selection introduces no host-device synchronization on the per-frame path.

A stale, missing, or inaccurate observation can only select a suboptimal decomposition; it cannot change execution completeness. The observation enters only $q_i(f)$ of Eq. 10. Every worklist candidate still reads the current $n_{\mathcal{W}}$ from the device and completes every coordinate in the valid worklist prefix; different candidates change only the tile shape and the reduction decomposition. The dense candidate forwards the same M_{out}, worklist, and device-resident length metadata untouched and updates every grid position, a superset of the scheduled ones, so it too covers $\mathcal{A}^i$. Candidate selection therefore satisfies the coverage contract whichever path it picks.

Learned-gating models lack the shared global signal the loop depends on. Their network input stays dense and each gated block predicts its own mask from its own features, so the count the loop would read is constant across frames and carries no information about any operator's worklist length. WISEConv assigns such a model fixed per-operator levels calibrated once from its gate statistics, rather than tracking each operator's own length, which would replace one readback per frame with one per operator. Only the configuration-selection policy changes; tile-local construction, cross-operator reuse, persistent worklist-driven compute, and the candidate set itself are unchanged.

4 Implementation

We implement WISEConv as a CUDA C++ extension for PyTorch. The current backend executes FP32 tensors in channels-last layout and provides the mask-aware operators required by the evaluated networks: convolution, transposed convolution, pooling, pointwise activation, addition, concatenation, and upsampling. A model-conversion pass replaces compatible PyTorch modules while preserving the graph topology, convolution parameters, and learned weights. The upstream application continues to generate masks.

The construction kernel assigns contiguous output positions to each warp. A thread tests one receptive field and terminates once it finds an active input. A warp ballot identifies active lanes, population counts give their local ranks, and one atomic operation reserves a contiguous worklist segment for the warp. Active lanes then write their coordinates in lane order. This realizes the tile-local ordering in Section 3.2 without a device-wide prefix scan or sorting pass. An identity 1×1 convolution has the same input and output coordinate space; when its input already carries compatible mask and worklist metadata, the operator reuses that worklist and skips construction.

The compute kernel uses a persistent, tiled implicit-GEMM organization. It keeps partial sums in registers and stages input and weight tiles through shared memory. Supported architectures use asynchronous global-to-shared copies, with a synchronous path for earlier GPUs. Split-K exposes additional parallel work when the active-coordinate dimension alone cannot occupy the device. Transposed convolution follows the same output-driven organization, but partitions output coordinates by stride phase so that each phase visits only its valid kernel offsets.

Efficient scheduling depends jointly on the GPU, layer shape, and activity. We therefore autotune the sparse tile shape and split-K factor and cache the choice by device, convolution parameters, activity level, and execution policy. For layers whose state-update semantics permit dense evaluation, the candidate set can also include a cuDNN path. A sparse-only policy isolates the worklist-driven kernel, and the cache key distinguishes the two policies and the requested determinism mode. Autotuning and one-time weight preparation complete before the timed sequence described in Section 5.1.

For fixed input values, weights, and mask, WISEConv evaluates Eq. (2) at every active output coordinate. Floating-point reordering may produce small numerical differences from cuDNN, so we verify active outputs with numerical tolerances rather than requiring bitwise identity. This check separates backend correctness from the task-quality effect of the fixed upstream mask policy.

5 Evaluation

5.1 Experimental Setup

We evaluate WISEConv across four perception models, three mask sources, four GPU platforms, and three competing execution paradigms. The evaluation asks four questions: (1) does WISEConv reduce full-model latency relative to the fastest available path, (2) how effectively does it convert required-work reduction into latency reduction, (3) what overhead does worklist construction add, and (4) does replacing the masked-convolution backend change task output or accuracy?

5.1.1 Platforms. We evaluate four representative GPU platforms: two embedded SoCs (Jetson Xavier NX and Jetson AGX Orin), a laptop-class mobile GPU (RTX 4070 Laptop), and a desktop GPU (RTX 3080). These devices cover the hardware classes commonly used for onboard deployment and robotics development. The two Jetson systems anchor our target deployment setting, while the discrete GPUs test whether WISEConv generalizes beyond embedded SoCs.

5.1.2 Models, Tasks, and Mask Sources. Our selected workloads span three robotic perception tasks:

- **Event-based optical flow.** We evaluate FireFlowNet [25], a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy, on MVSEC [42]. MVSEC contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [7], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks. We calibrate the sparsification policy on `indoor_flying1` and evaluate on the other three sequences.
- **Video object detection.** We evaluate YOLOv8n and YOLOv8m [18] on MOT16 [23], a collection of crowded pedestrian videos with annotated person boxes. Both models produce bounding boxes and confidence scores, while their different scales expose how model size affects sparse-execution efficiency. DeltaCNN [27] supplies the temporal mask mechanism by propagating thresholded activation deltas between successive frames.
- **Human pose estimation.** We use the DynConv pose model [34] on MPII [1], which depicts people performing diverse activities and annotates their 2D body joints. The model predicts joint heatmaps; its learned spatial gates provide input-dependent masks at gated blocks.

Table 1 summarizes their models and evaluation scales. We use official weights for all four models [18, 25, 34]. For each workload, we fix these weights, inputs, and upstream mask policy; the sparse paths receive the same resulting layer masks and differ only in their masked-convolution backend.

Table 1. Evaluation workloads.

	FireFlowNet [25]	**YOLOv8n/m** [18]	**DynConv Pose** [34]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [7]	DeltaCNN [27]	DynConv [34]
Dataset	MVSEC [42]	MOT16 [23]	MPII [1]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

5.1.3 Execution Baselines. We compare WISEConv against three FP32 execution paths established by prior work.

- **Dense** uses the native PyTorch/cuDNN backend [5] with sparse execution disabled.
- **Tile skipping** follows the tile-sparse execution established by SBNet, Ev-Conv, and DeltaCNN [7, 27, 29].
- **Gather-scatter** uses the original DynConv CUDA backend for human pose [34] and an implementation equivalent to MMCV `MaskedConv2d` for FireFlowNet and YOLO [4].

5.1.4 Metrics and Methodology. Our primary metric is full-model GPU latency. The timed region includes mask propagation, worklist construction, all network operators, and output update, but excludes host-to-device input transfer, calibration, autotuning, and the one-time dense state initialization required by incremental models. We use CUDA events on the model's compute stream and process complete held-out sequences rather than isolated synthetic frames. Each sequence is repeated three times; we retain per-frame samples and report both aggregate latency and dispersion.

To explain latency, we report the required-work ratio ρ_{work}, useful-compute ratio η, convolution throughput, and

the construction fraction of operator latency. Throughput here isolates the compute kernel; construction is reported separately as a fraction of operator latency, so readers can reconstruct the inclusive T of §2.2. We aggregate η as total active FLOPs over total scheduled FLOPs across all convolution layers, so each layer contributes in proportion to the work it actually schedules. Because layers differ in cost, an unweighted average of layer activities misstates the work a mask removes, so we weight each layer by its dense MAC count,

$$\rho_{\text{work}} = \frac{\sum_l \alpha^l W^l_{\text{dense}}}{\sum_l W^l_{\text{dense}}}, \qquad \alpha^l = \frac{\|M^l\|_0}{H^l_{out} W^l_{out}}, \tag{12}$$

where W^l_{dense} is the dense MAC count of layer l. Task quality is measured with Average Endpoint Error (AEE) for optical flow, detection AP against the fixed evaluation references for YOLO, and PCKh for human pose. Accuracy comparisons use the same masks and operating point as latency measurements; they evaluate the upstream policy, while operator correctness is checked separately against dense convolution at every active output.

5.2 Full-Model Latency

5.3 Efficiency Analysis

5.4 Construction Overhead

5.5 Task Accuracy

References

[1] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[2] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[3] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[5] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[6] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[7] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. EvConv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[8] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[9] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[10] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[11] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[12] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[13] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[14] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[15] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[16] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[17] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[18] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[19] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[20] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[21] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[22] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[23] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking.

CoRR abs/1603.00831 (2016).

[24] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[25] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[26] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[27] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[28] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[29] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[30] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[31] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[32] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[33] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[34] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[35] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[36] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[37] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[38] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[39] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[40] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[41] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[42] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).